

Manual imprimible

MediaPipe FaceMesh legacy y Face Landmarker actual

Uso en local y en Google Colab

1. Objetivo del manual

Este manual explica cómo usar dos formas de detección de landmarks faciales con MediaPipe:

1. **FaceMesh legacy**, usando:

```
mp.solutions.face_mesh.FaceMesh
```

1. **Face Landmarker actual**, usando:

```
mp.tasks.vision.FaceLandmarker
```

El objetivo es tener una guía práctica para trabajar en:

- entorno local con Python 3.10;
 - Google Colab;
 - imagen fija;
 - webcam local.
-

2. Diferencia general entre FaceMesh legacy y Face Landmarker actual

Opción	API	Estado	Uso recomendado
FaceMesh legacy	<code>mp.solutions.face_mesh.FaceMesh</code>	API antigua / legacy	Tutoriales viejos, clases introductorias, código existente
Face Landmarker actual	<code>mp.tasks.vision.FaceLandmarker</code>	API nueva de MediaPipe Tasks	Proyectos nuevos, código más actual, blendshapes, expresiones faciales

FaceMesh legacy fue durante años la forma más común de trabajar con malla facial en MediaPipe. Muchos tutoriales de YouTube y notebooks antiguos usan esta versión.

Face Landmarker es la versión más actual dentro de MediaPipe Tasks. Requiere descargar un modelo `.task` y configurar el detector con una estructura diferente.

3. Recomendación general

Para una clase introductoria, conviene explicar primero **FaceMesh legacy**, porque el código es más simple y ayuda a entender el concepto de landmarks faciales.

Para un proyecto nuevo, conviene usar **Face Landmarker**, porque es la API actual de MediaPipe.

Situación	Recomendación
Quiero seguir tutoriales antiguos	FaceMesh legacy
Quiero explicar landmarks de forma simple	FaceMesh legacy
Quiero hacer un proyecto nuevo	Face Landmarker
Quiero usar MediaPipe Tasks	Face Landmarker
Quiero detectar expresiones faciales / blendshapes	Face Landmarker
Quiero usar código corto para clase	FaceMesh legacy
Quiero código más alineado con documentación actual	Face Landmarker

PARTE 1

FaceMesh legacy en entorno local

4. Qué es FaceMesh legacy

FaceMesh legacy es la solución clásica de MediaPipe para detectar puntos del rostro.

Se usa desde:

```
mp.solutions.face_mesh
```

y permite obtener una malla facial con puntos de referencia sobre ojos, nariz, boca, contorno de la cara y otras zonas.

Ejemplo de inicialización:

```
import mediapipe as mp

mp_face_mesh = mp.solutions.face_mesh

face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
)
```

5. Requisitos para FaceMesh legacy en local

Se recomienda usar:

- Python 3.10;
- entorno virtual;
- MediaPipe 0.10.14;
- OpenCV;
- NumPy menor a 2;
- Matplotlib.

6. Crear carpeta del proyecto

En Windows, abrir una terminal y ejecutar:

```
mkdir proyecto_mediapipe_legacy
cd proyecto_mediapipe_legacy
```

7. Crear entorno virtual con Python 3.10

En Windows:

```
py -3.10 -m venv .venv
```

Activar el entorno:

```
.venv\Scripts\activate
```

En Linux o Mac:

```
python3.10 -m venv .venv
source .venv/bin/activate
```

8. Crear archivo `requirements.txt`

Crear un archivo llamado:

```
requirements.txt
```

con este contenido:

```
mediapipe==0.10.14
opencv-python==4.10.0.84
numpy<2
matplotlib
```

9. Instalar dependencias

Con el entorno virtual activado:

```
pip install -r requirements.txt
```

10. Verificar instalación

Ejecutar:

```
python -c "import mediapipe as mp; import cv2; print('MediaPipe:',
mp.__version__); print('OpenCV:', cv2.__version__)"
```

Si muestra las versiones sin error, la instalación está funcionando.

11. Estructura recomendada del proyecto

```
proyecto_mediapipe_legacy/
|
├─ .venv/
├─ requirements.txt
```

```
|— main_facemesh_legacy.py
|— rostro.jpg
```

La imagen `rostro.jpg` debe estar en la misma carpeta que el archivo `.py`.

12. Código FaceMesh legacy con imagen

Crear el archivo:

```
main_facemesh_legacy.py
```

Código:

```
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt

# Inicializamos módulos legacy de MediaPipe
mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

# Cargamos imagen con OpenCV
imagen_bgr = cv2.imread("rostro.jpg")

if imagen_bgr is None:
    raise FileNotFoundError("No se encontró la imagen rostro.jpg")

# OpenCV carga en BGR.
# MediaPipe trabaja con RGB.
imagen_rgb = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2RGB)

# Creamos el detector FaceMesh legacy
with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:

    resultado = face_mesh.process(imagen_rgb)

    imagen_anotada = imagen_rgb.copy()

    if resultado.multi_face_landmarks:
        for face_landmarks in resultado.multi_face_landmarks:
```

```

        # Dibuja la malla completa
        mp_drawing.draw_landmarks(
            image=imagen_anotada,
            landmark_list=face_landmarks,
            connections=mp_face_mesh.FACEMESH_TESSELATION,
            landmark_drawing_spec=None,

connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_tesselation_style()
        )

        # Dibuja los contornos principales
        mp_drawing.draw_landmarks(
            image=imagen_anotada,
            landmark_list=face_landmarks,
            connections=mp_face_mesh.FACEMESH_CONTOURS,
            landmark_drawing_spec=None,

connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_contours_style()
        )

    else:
        print("No se detectó rostro.")

plt.figure(figsize=(8, 8))
plt.imshow(imagen_anotada)
plt.axis("off")
plt.show()

```

13. Ejecutar el script

En la terminal:

```
python main_facemesh_legacy.py
```

14. FaceMesh legacy con webcam local

Crear el archivo:

```
webcam_facemesh_legacy.py
```

Código:

```

import cv2
import mediapipe as mp

```

```

mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

cap = cv2.VideoCapture(0)

if not cap.isOpened():
    raise RuntimeError("No se pudo abrir la cámara.")

with mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
) as face_mesh:

    while True:
        ret, frame_bgr = cap.read()

        if not ret:
            print("No se pudo leer frame de la cámara.")
            break

        frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)

        resultado = face_mesh.process(frame_rgb)

        if resultado.multi_face_landmarks:
            for face_landmarks in resultado.multi_face_landmarks:
                mp_drawing.draw_landmarks(
                    image=frame_bgr,
                    landmark_list=face_landmarks,
                    connections=mp_face_mesh.FACEMESH_CONTOURS,
                    landmark_drawing_spec=None,

connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_contours_style()
                )

                cv2.imshow("FaceMesh legacy", frame_bgr)

                # Presionar ESC para salir
                if cv2.waitKey(1) & 0xFF == 27:
                    break

cap.release()
cv2.destroyAllWindows()

```

Ejecutar:

```
python webcam_facemesh_legacy.py
```

PARTE 2

FaceMesh legacy en Google Colab

15. Consideración importante sobre Colab

En Google Colab puede pasar que ya existan versiones cargadas de `numpy`, `opencv-python`, `opencv-python-headless` o `mediapipe`.

Por eso, si se instalan versiones específicas, conviene:

1. desinstalar versiones previas;
2. instalar las versiones necesarias;
3. reiniciar el entorno de ejecución;
4. volver a ejecutar el notebook desde arriba.

Esto evita conflictos entre paquetes ya cargados en memoria y paquetes recién instalados.

16. Instalación limpia de FaceMesh legacy en Colab

Primera celda:

```
!pip uninstall -y mediapipe opencv-python opencv-python-headless numpy
!pip install mediapipe==0.10.14 opencv-python==4.10.0.84 "numpy<2" matplotlib
```

17. Reiniciar entorno de ejecución en Colab

Después de instalar:

```
import os
os.kill(os.getpid(), 9)
```

El entorno se va a reiniciar. Luego hay que volver a ejecutar el notebook desde la primera celda de imports.

18. Verificar instalación después del reinicio

```
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt

print("MediaPipe:", mp.__version__)
print("OpenCV:", cv2.__version__)
```

19. Subir imagen a Colab

```
from google.colab import files

uploaded = files.upload()
```

Subir una imagen llamada:

rostro.jpg

20. Código completo FaceMesh legacy en Colab

```
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt

mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

ruta_imagen = "rostro.jpg"

imagen_bgr = cv2.imread(ruta_imagen)

if imagen_bgr is None:
    raise FileNotFoundError("No se encontró la imagen. Revisá el nombre del archivo.")

imagen_rgb = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2RGB)

with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
```

```

        min_detection_confidence=0.5
    ) as face_mesh:

        resultado = face_mesh.process(imagen_rgb)

        imagen_annotada = imagen_rgb.copy()

        if resultado.multi_face_landmarks:
            for face_landmarks in resultado.multi_face_landmarks:
                mp_drawing.draw_landmarks(
                    image=imagen_annotada,
                    landmark_list=face_landmarks,
                    connections=mp_face_mesh.FACEMESH_TESSELATION,
                    landmark_drawing_spec=None,

connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_tesselation_style()
                )

                mp_drawing.draw_landmarks(
                    image=imagen_annotada,
                    landmark_list=face_landmarks,
                    connections=mp_face_mesh.FACEMESH_CONTOURS,
                    landmark_drawing_spec=None,

connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_contours_style()
                )
            else:
                print("No se detectó rostro.")

plt.figure(figsize=(8, 8))
plt.imshow(imagen_annotada)
plt.axis("off")
plt.show()

```

PARTE 3

Face Landmarker actual en entorno local

21. Qué es Face Landmarker

Face Landmarker es la tarea actual de MediaPipe para detectar landmarks faciales.

Pertenece a MediaPipe Tasks y permite trabajar con:

- imágenes;

- videos;
- stream en tiempo real;
- landmarks faciales;
- expresiones faciales;
- blendshapes;
- matrices de transformación facial.

A diferencia de FaceMesh legacy, Face Landmarker necesita descargar un archivo de modelo `.task`.

22. Crear carpeta del proyecto

```
mkdir proyecto_face_landmarker  
cd proyecto_face_landmarker
```

23. Crear entorno virtual con Python 3.10

En Windows:

```
py -3.10 -m venv .venv  
.venv\Scripts\activate
```

En Linux o Mac:

```
python3.10 -m venv .venv  
source .venv/bin/activate
```

24. Crear `requirements.txt`

Crear un archivo llamado:

```
requirements.txt
```

con este contenido:

```
mediapipe  
opencv-python  
matplotlib
```

25. Instalar dependencias

```
pip install -r requirements.txt
```

26. Estructura recomendada del proyecto Face Landmarker

```
proyecto_face_landmarker/  
├── .venv/  
├── requirements.txt  
├── main_face_landmarker.py  
├── webcam_face_landmarker.py  
├── rostro.jpg  
└── modelos/  
    └── face_landmarker.task
```

27. Descargar el modelo `.task`

Crear archivo:

```
descargar_modelo.py
```

Código:

```
import urllib.request  
from pathlib import Path  
  
carpeta_modelos = Path("modelos")  
carpeta_modelos.mkdir(exist_ok=True)  
  
url_modelo = "https://storage.googleapis.com/mediapipe-models/  
face_landmarker/face_landmarker/float16/latest/face_landmarker.task"  
ruta_modelo = carpeta_modelos / "face_landmarker.task"  
  
urllib.request.urlretrieve(url_modelo, ruta_modelo)  
  
print("Modelo descargado en:", ruta_modelo)
```

Ejecutar:

```
python descargar_modelo.py
```

28. Código Face Landmarker actual con imagen

Crear archivo:

```
main_face_landmarker.py
```

Código:

```
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt

BaseOptions = mp.tasks.BaseOptions
FaceLandmarker = mp.tasks.vision.FaceLandmarker
FaceLandmarkerOptions = mp.tasks.vision.FaceLandmarkerOptions
VisionRunningMode = mp.tasks.vision.RunningMode

ruta_modelo = "modelos/face_landmarker.task"
ruta_imagen = "rostro.jpg"

options = FaceLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=ruta_modelo),
    running_mode=VisionRunningMode.IMAGE,
    num_faces=1,
    output_face_blendshapes=True,
    output_facial_transformation_matrixes=True
)

mp_image = mp.Image.create_from_file(ruta_imagen)

with FaceLandmarker.create_from_options(options) as landmarker:
    resultado = landmarker.detect(mp_image)

print("Cantidad de rostros detectados:", len(resultado.face_landmarks))

imagen_bgr = cv2.imread(ruta_imagen)

if imagen_bgr is None:
    raise FileNotFoundError("No se encontró la imagen.")

imagen_rgb = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2RGB)

alto, ancho, _ = imagen_rgb.shape
```

```

if resultado.face_landmarks:
    for rostro in resultado.face_landmarks:
        for punto in rostro:
            x = int(punto.x * ancho)
            y = int(punto.y * alto)
            cv2.circle(imagen_rgb, (x, y), 1, (0, 255, 0), -1)
else:
    print("No se detectó rostro.")

plt.figure(figsize=(8, 8))
plt.imshow(imagen_rgb)
plt.axis("off")
plt.show()

```

Ejecutar:

```
python main_face_landmarker.py
```

29. Face Landmarker actual con webcam local

Crear archivo:

```
webcam_face_landmarker.py
```

Código:

```

import time
import cv2
import mediapipe as mp

BaseOptions = mp.tasks.BaseOptions
FaceLandmarker = mp.tasks.vision.FaceLandmarker
FaceLandmarkerOptions = mp.tasks.vision.FaceLandmarkerOptions
VisionRunningMode = mp.tasks.vision.RunningMode

ruta_modelo = "modelos/face_landmarker.task"

options = FaceLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=ruta_modelo),
    running_mode=VisionRunningMode.VIDEO,
    num_faces=1
)

cap = cv2.VideoCapture(0)

if not cap.isOpened():

```

```

raise RuntimeError("No se pudo abrir la cámara.")

with FaceLandmarker.create_from_options(options) as landmarker:

    while True:
        ret, frame_bgr = cap.read()

        if not ret:
            print("No se pudo leer frame.")
            break

        frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)

        mp_image = mp.Image(
            image_format=mp.ImageFormat.SRGB,
            data=frame_rgb
        )

        timestamp_ms = int(time.time() * 1000)

        resultado = landmarker.detect_for_video(mp_image, timestamp_ms)

        alto, ancho, _ = frame_bgr.shape

        if resultado.face_landmarks:
            for rostro in resultado.face_landmarks:
                for punto in rostro:
                    x = int(punto.x * ancho)
                    y = int(punto.y * alto)
                    cv2.circle(frame_bgr, (x, y), 1, (0, 255, 0), -1)

        cv2.imshow("Face Landmarker actual", frame_bgr)

        # Presionar ESC para salir
        if cv2.waitKey(1) & 0xFF == 27:
            break

    cap.release()
    cv2.destroyAllWindows()

```

Ejecutar:

```
python webcam_face_landmarker.py
```

PARTE 4

Face Landmarker actual en Google Colab

30. Consideración importante sobre Colab

Para Face Landmarker actual también conviene hacer una instalación limpia si el notebook ya tenía paquetes cargados.

El procedimiento recomendado es:

1. desinstalar paquetes previos;
 2. instalar paquetes actuales;
 3. reiniciar entorno;
 4. ejecutar el notebook desde arriba.
-

31. Instalación limpia en Colab

Primera celda:

```
!pip uninstall -y mediapipe opencv-python opencv-python-headless numpy
!pip install mediapipe opencv-python matplotlib
```

32. Reiniciar entorno de ejecución

```
import os
os.kill(os.getpid(), 9)
```

Después del reinicio, volver a ejecutar desde la primera celda de imports.

33. Verificar instalación

```
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt

print("MediaPipe:", mp.__version__)
print("OpenCV:", cv2.__version__)
```

34. Descargar modelo `.task` en Colab

```
import urllib.request
from pathlib import Path

Path("modelos").mkdir(exist_ok=True)

url_modelo = "https://storage.googleapis.com/mediapipe-models/
face_landmarker/face_landmarker/float16/latest/face_landmarker.task"
ruta_modelo = "modelos/face_landmarker.task"

urllib.request.urlretrieve(url_modelo, ruta_modelo)

print("Modelo descargado:", ruta_modelo)
```

35. Subir una imagen

```
from google.colab import files

uploaded = files.upload()
```

Subir una imagen llamada:

```
rostro.jpg
```

36. Código completo Face Landmarker actual en Colab

```
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt

BaseOptions = mp.tasks.BaseOptions
FaceLandmarker = mp.tasks.vision.FaceLandmarker
FaceLandmarkerOptions = mp.tasks.vision.FaceLandmarkerOptions
VisionRunningMode = mp.tasks.vision.RunningMode

ruta_modelo = "modelos/face_landmarker.task"
ruta_imagen = "rostro.jpg"

options = FaceLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=ruta_modelo),
    running_mode=VisionRunningMode.IMAGE,
```

```

        num_faces=1,
        output_face_blendshapes=True,
        output_facial_transformation_matrixes=True
    )

    mp_image = mp.Image.create_from_file(ruta_imagen)

    with FaceLandmarker.create_from_options(options) as landmarker:
        resultado = landmarker.detect(mp_image)

    print("Cantidad de rostros detectados:", len(resultado.face_landmarks))

    imagen_bgr = cv2.imread(ruta_imagen)

    if imagen_bgr is None:
        raise FileNotFoundError("No se encontró la imagen.")

    imagen_rgb = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2RGB)

    alto, ancho, _ = imagen_rgb.shape

    if resultado.face_landmarks:
        for rostro in resultado.face_landmarks:
            for punto in rostro:
                x = int(punto.x * ancho)
                y = int(punto.y * alto)
                cv2.circle(imagen_rgb, (x, y), 1, (0, 255, 0), -1)
    else:
        print("No se detectó rostro.")

    plt.figure(figsize=(8, 8))
    plt.imshow(imagen_rgb)
    plt.axis("off")
    plt.show()

```

PARTE 5

Comparación de código

37. Importación

FaceMesh legacy:

```
import mediapipe as mp

mp_face_mesh = mp.solutions.face_mesh
```

Face Landmarker actual:

```
import mediapipe as mp

FaceLandmarker = mp.tasks.vision.FaceLandmarker
FaceLandmarkerOptions = mp.tasks.vision.FaceLandmarkerOptions
```

38. Creación del detector

FaceMesh legacy:

```
with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:
    resultado = face_mesh.process(imagen_rgb)
```

Face Landmarker actual:

```
options = FaceLandmarkerOptions(
    base_options=BaseOptions(model_asset_path="modelos/
face_landmarker.task"),
    running_mode=VisionRunningMode.IMAGE
)

with FaceLandmarker.create_from_options(options) as landmarker:
    resultado = landmarker.detect(mp_image)
```

39. Resultado

FaceMesh legacy:

```
resultado.multi_face_landmarks
```

Face Landmarker actual:

```
resultado.face_landmarks
```

40. Imagen de entrada

FaceMesh legacy trabaja directamente con un array RGB:

```
imagen_rgb = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2RGB)
resultado = face_mesh.process(imagen_rgb)
```

Face Landmarker usa un objeto `mp.Image`:

```
mp_image = mp.Image.create_from_file("rostro.jpg")
resultado = landmarker.detect(mp_image)
```

41. Video o webcam

FaceMesh legacy usa:

```
resultado = face_mesh.process(frame_rgb)
```

Face Landmarker actual usa:

```
resultado = landmarker.detect_for_video(mp_image, timestamp_ms)
```

Además, Face Landmarker necesita configurar:

```
running_mode=VisionRunningMode.VIDEO
```

PARTE 6

Errores frecuentes

42. Error: no se encuentra la imagen

Mensaje posible:

```
FileNotFoundError: No se encontró la imagen rostro.jpg
```

Solución:

- revisar que la imagen esté en la misma carpeta que el script;
- revisar que el nombre sea exactamente `rostro.jpg` ;
- revisar mayúsculas, minúsculas y extensión.

43. Error con OpenCV y `cv2.imshow`

Mensaje posible:

```
The function is not implemented
```

Puede pasar si se instaló:

```
opencv-python-headless
```

Para usar ventanas locales con `cv2.imshow` , conviene instalar:

```
opencv-python
```

No usar `opencv-python-headless` en proyectos locales con ventana.

44. Error por versión de NumPy

Si aparecen errores relacionados con NumPy, en FaceMesh legacy conviene usar:

```
numpy<2
```

Por eso el `requirements.txt` recomendado para FaceMesh legacy es:

```
mediapipe==0.10.14  
opencv-python==4.10.0.84  
numpy<2  
matplotlib
```

45. Error en Colab después de instalar

Si instalaste y sigue dando error, reiniciar el entorno:

```
import os
os.kill(os.getpid(), 9)
```

Después ejecutar todo desde arriba.

46. Error: Python muy nuevo

Para entorno local, evitar versiones muy nuevas de Python si se trabaja con MediaPipe legacy.

Usar Python 3.10:

```
py -3.10 -m venv .venv
```

PARTE 7

Recomendación para clase

47. Secuencia didáctica sugerida

Para explicar el tema en clase:

1. Explicar qué es un landmark facial.
 2. Mostrar una imagen de un rostro.
 3. Explicar que MediaPipe detecta puntos sobre el rostro.
 4. Mostrar FaceMesh legacy porque el código es más directo.
 5. Mostrar que FaceMesh legacy pertenece a la API antigua.
 6. Explicar que la API actual es Face Landmarker.
 7. Comparar ambos códigos.
 8. Cerrar con una recomendación práctica: legacy para entender, Face Landmarker para proyectos nuevos.
-

48. Explicación simple para estudiantes

Un landmark facial es un punto de referencia en el rostro.

Por ejemplo:

- puntos alrededor de los ojos;
- puntos sobre la boca;
- puntos en la nariz;
- puntos en el contorno de la cara.

MediaPipe detecta esos puntos y permite usarlos para analizar gestos, orientación del rostro, parpadeos, posición de la boca o dirección aproximada de la cabeza.

FaceMesh legacy fue la forma clásica de hacerlo. Face Landmarker es la forma más actual.

49. Qué opción usar en cada caso

Caso	Usar
Notebook simple para clase	FaceMesh legacy
Proyecto nuevo	Face Landmarker
Código viejo de tutoriales	FaceMesh legacy
API actual de MediaPipe	Face Landmarker
Webcam local simple	FaceMesh legacy o Face Landmarker
Colab con imagen	FaceMesh legacy o Face Landmarker
Colab con webcam	No es lo más recomendable para empezar

50. Conclusión

FaceMesh legacy sigue sirviendo para aprender y para adaptar tutoriales antiguos. Es más simple de explicar porque usa directamente `mp.solutions.face_mesh`.

Face Landmarker es la opción más actual para proyectos nuevos. Su estructura es un poco más larga porque usa MediaPipe Tasks y requiere un modelo `.task`, pero está mejor alineada con la documentación actual.

Para una clase, se puede enseñar primero FaceMesh legacy y luego mostrar Face Landmarker como evolución actual.

Fuentes

- MediaPipe Face Mesh legacy:
https://github.com/google-ai-edge/mediapipe/blob/master/docs/solutions/face_mesh.md

- MediaPipe Face Mesh en Read the Docs:
https://mediapipe.readthedocs.io/en/latest/solutions/face_mesh.html
- MediaPipe Face Landmarker — documentación oficial:
https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker
- MediaPipe Face Landmarker — guía Python:
https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/python
- MediaPipe en PyPI:
<https://pypi.org/project/mediapipe/>
- Ejemplo oficial de Face Landmarker en notebook:
https://github.com/googlesamples/mediapipe/blob/main/examples/face_landmarker/python/%5BMediaPipe_Python_Tasks%5D_Face_Landmarker.ipynb
- Modelo Face Landmarker `.task` :
https://storage.googleapis.com/mediapipe-models/face_landmarker/face_landmarker/float16/latest/face_landmarker.task